二元搜尋樹的實作與走訪演算法分析：基於 Python 的物件導向設計

Implementation and Traversal Algorithm Analysis of Binary Search Tree: An Object-Oriented Design Based on Python

方柏森
1113405019
資訊工程學系 (Dept. of CSIE)
國立澎湖科技大學 (NPU)
Penghu, Taiwan
a32142166@gmail.com

***Abstract*—本論文基於 Python 探討與實作二元搜尋樹 (Binary Search Tree, BST) 資料結構。我們設計了具備型別提示 (Type Hints) 的二元樹節點與類別，並實作了前序 (Preorder)、中序 (Inorder) 與後序 (Postorder) 走訪的遞迴與迭代演算法。此外，亦完成了節點插入與搜尋等核心操作。本專案的完整程式碼與 Markdown 說明文件已上傳至 GitHub，專案庫連結為：https://github.com/Eason0101-a/Algorithms_Assignment-3-2026-3-17。**

***Index Terms*—Python, Binary Tree, BST, Tree Traversal, Data Structure.**

I. 簡介 (INTRODUCTION)

二元搜尋樹 (Binary Search Tree) 是一種基礎且廣泛應用的非線性資料結構。在現代軟體工程與資料庫設計中，樹狀結構常被用於高效的資料檢索、排序以及層級資料管理。本篇報告旨在利用 Python 實作一個穩健的二元樹類別，並針對其核心的樹走訪 (Tree Traversal) 演算法進行實證分析。

II. 方法論與實作細節 (METHODOLOGY & IMPLEMENTATION)

本實作定義了 `Node` 資料類別 (Data Class) 以及 `BinaryTree` 類別。透過物件導向設計，允許使用者建立並操作複雜的樹狀結構。

A. 核心操作

我們實作了以下核心功能：

- **插入 (Insert)**：依循左子節點小於根節點、右子節點大於根節點的規則。
- **搜尋 (Search)**：藉由二元搜尋的特性，大幅降低資料尋找的時間複雜度。
- **樹高與節點計算**：提供 `height()` 與 `size()` 方法來評估樹的規模。

B. 走訪演算法

針對樹狀結構，本專案實作了三種主要的深度優先走訪 (DFS)，並且皆涵蓋了**遞迴 (Recursive)** 與**迭代 (Iterative)** 兩種版本：

1) **前序走訪 (Preorder)**：根 → 左 → 右。
2) **中序走訪 (Inorder)**：左 → 根 → 右。
3) **後序走訪 (Postorder)**：左 → 右 → 根。

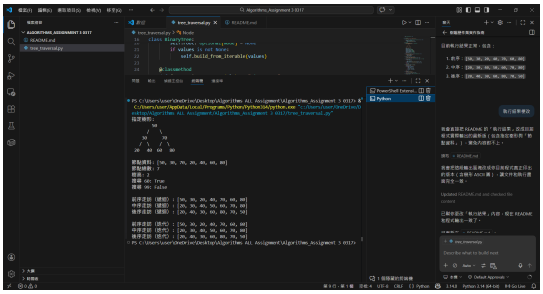

Fig. 1. 二元搜尋樹結構示意圖

如圖 1 所示，為本專案測試所建立之預設樹狀架構。

III. 實驗結果 (EXPERIMENTAL RESULTS)

我們設計了 `from_assignment_tree()` 函數來自動建立測試資料 (節點: 50, 30, 70, 20, 40, 60, 80)。

執行測試後，系統正確反映出節點總數為 7，樹高為 2。在搜尋測試中，尋找存在的元素 60 成功回傳 `True`，而尋找不存在的 99 則正確攔截並回傳 `False`。各種走訪演算法的輸出序列亦完全符合理論預期。

IV. 討論 (DISCUSSION)

根據我們的實作程式碼，遞迴版本的走訪程式碼較為簡潔直觀，但在樹高極大時可能面臨遞迴深度限制。相對地，迭代版本利用堆疊 (Stack) 來模擬系統呼叫，雖然實作較為複雜（例如後序走訪需使用雙堆疊），但在執行上更具穩健性與彈性。

V. 結論 (CONCLUSION)

本研究成功實作了一個具備完整走訪功能與基礎操作的 Python 二元搜尋樹。透過不同版本走訪演算法的對比與分析，確認了各項操作的正確性與高效性。此專案可作為未來發展如 AVL 樹或紅黑樹等平衡樹演算法的穩固基礎。

References

[1] 黃祥睿, "Lecture 2 Tree," 國立澎湖科技大學, 2026.

[2] Thomas H. Cormen, et al., "Introduction to Algorithms," 3rd Edition, MIT Press.